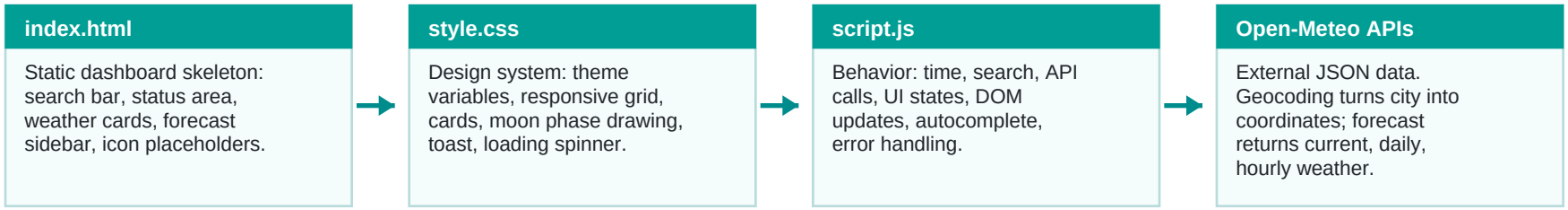
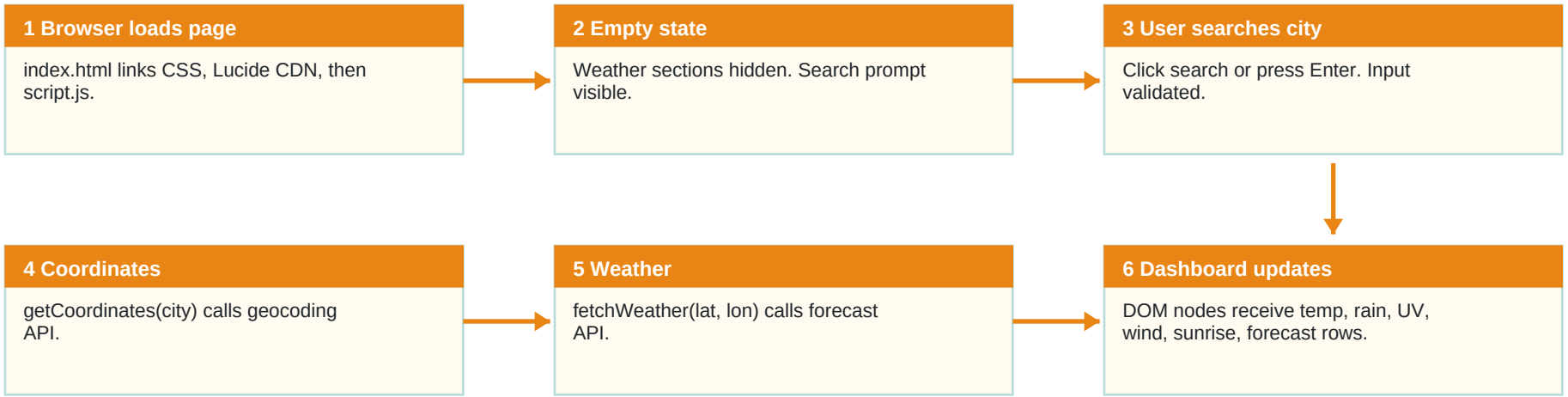
Weather Dashboard: Visual Code Guide

How this repo works: HTML gives the app bones, CSS gives the look, JavaScript connects user actions to live weather data.

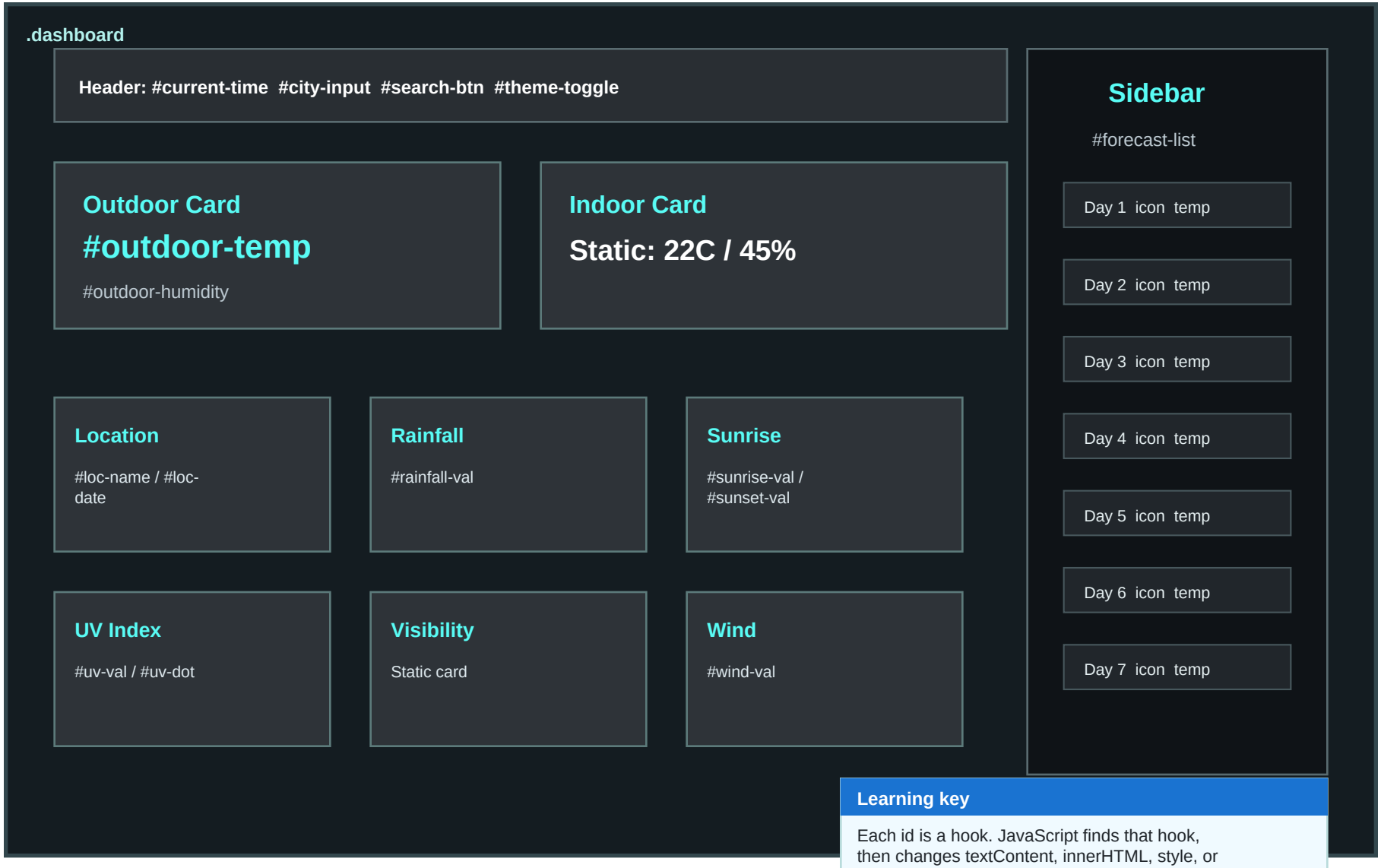


Main Runtime Pipeline



Page Structure: What User Sees

index.html creates named placeholders. script.js fills those placeholders after API data arrives. style.css makes the layout responsive.



Learning key

Each id is a hook. JavaScript finds that hook, then changes textContent, innerHTML, style, or data-lucide.

JavaScript: State Machine + Function Map

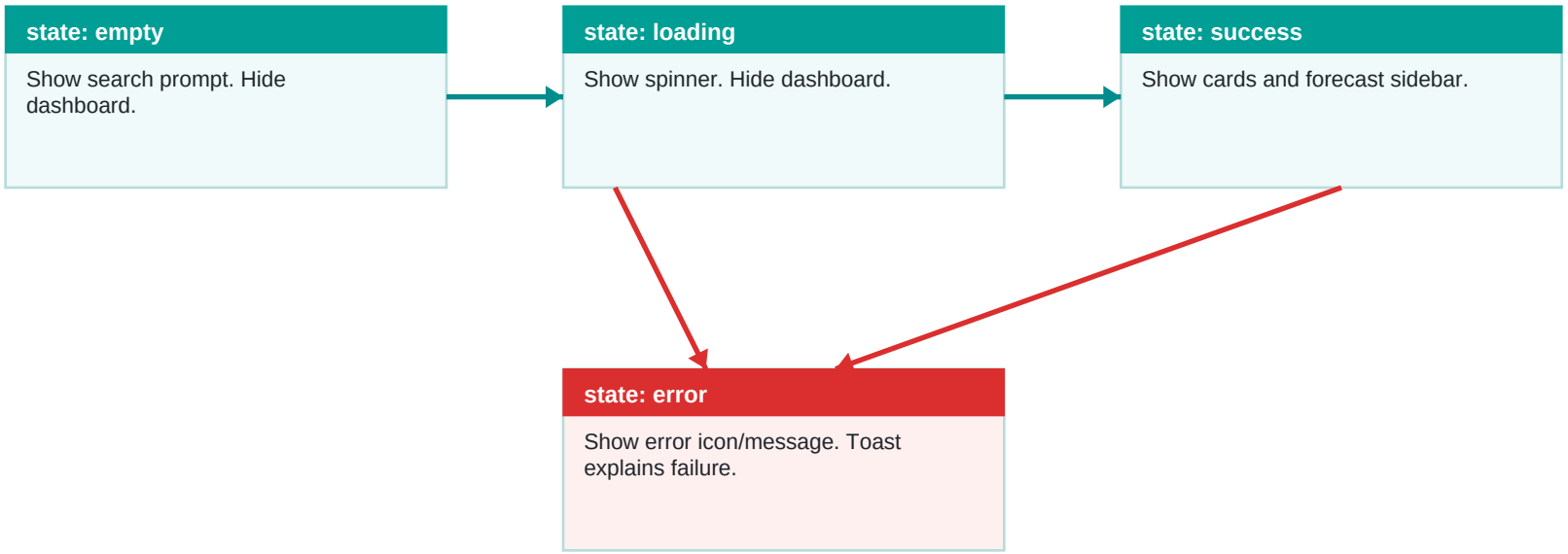
script.js is small because one function, setUIState(), controls which screen is visible. Fetch functions only gather data; DOM updates happen after data is valid.

Autocomplete

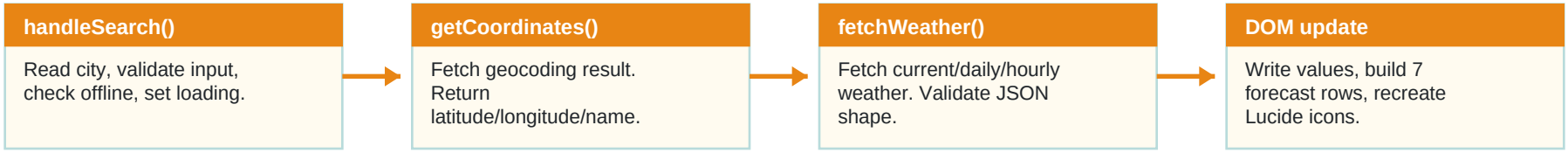
input event starts 300ms debounce. fetchSuggestions() asks geocoding API for 5 matches. showSuggestions() creates clickable rows.

Theme

Click toggles body.light-mode. CSS variables do most visual work. Icon switches sun/moon.

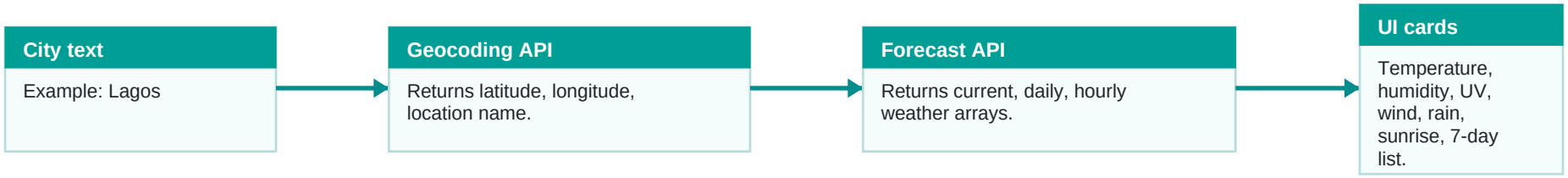


Search Path



Core Concepts Behind Code

API Data Map



CSS Learning Map

CSS variables

:root dark theme. body.light-mode overrides same names.

Grid

.top-section and .bottom-section place dashboard cards.

Flexbox

Header, cards, sidebar, forecast rows, status center.

Media queries

1100px, 768px, 480px reshape layout for small screens.

Pure CSS visuals

Moon phase arc, compass, UV bar, spinner, toast.

Best Next Fixes

- Close final CSS @media block if missing in editor.
- Use textContent/createElement for suggestions to reduce XSS risk.
- Either display hourly data or stop requesting hourly fields.
- Make visibility dynamic or label it as static example data.

One sentence model

User input -> API JSON -> validated data -> DOM updates -> responsive dashboard.

script.js Piece by Piece (1/4): Boot, Clock, Theme, UI States

The clock (lines 11-21)

```
function updateTime() {
  const now = new Date();                      // current moment
  const timeStr = now.toLocaleTimeString('en-US',
      { hour: '2-digit', minute: '2-digit' })
      .replace(':', ' : ');                    // "04 : 32 PM"
  document.getElementById('current-time').textContent = timeStr;
}
setInterval(updateTime, 1000);   // re-run every second
updateTime();                    // and once at startup
// also writes 'SUNDAY / 7 JUL, 2026' into #loc-date each tick
```

Theme toggle (lines 24-30)

```
themeBtn.addEventListener('click', () => {
  document.body.classList.toggle('light-mode');  // CSS vars flip
  themeIcon.setAttribute('data-lucide',
    body.classList.contains('light-mode') ? 'moon' : 'sun');
  lucide.createIcons({ nameAttr: 'data-lucide' }); // redraw icon
});
```

setUIState() - one function owns the screen (lines 41-76)

```
function setUIState(state, message = '') {
  // 'empty'   -> hide cards, show search prompt
  // 'loading' -> hide cards, show spinning loader
  // 'error'   -> hide cards, show red cloud-off + message
  if (state === 'success') {
    statusContainer.style.display = 'none';
    weatherTop.style.display    = 'grid';
    weatherBottom.style.display = 'grid';
    sidebarWrapper.style.display = 'flex';
  }
  lucide.createIcons();  // re-render <i data-lucide> as SVG
}
```

Concept: polling loop

setInterval() calls a function on a timer. The clock is a 1-second loop: compute string, write into the DOM.

Concept: CSS does the theme

JS only toggles one class on <body>. All colors change because style.css redefines the same CSS variables under body.light-mode.

Concept: state machine

Every screen change goes through setUIState(). No scattered show/hide calls = impossible to show error and data at once.

showError() toast (lines 33-38)

Adds .show class to #error-toast, removes it after 4000ms. CSS transition does the slide animation.

script.js Piece by Piece (2/4): getCoordinates() - City Name to Lat/Lon

getCoordinates() (lines 79-104)

```
async function getCoordinates(city) {
  const geoUrl = 'https://geocoding-api.open-meteo.com/v1/search
                 ?name=${encodeURIComponent(city)}&count=1...';

  const controller = new AbortController();      // cancel handle
  const timeoutId = setTimeout(() => controller.abort(), 5000);

  try {
    const res = await fetch(geoUrl, { signal: controller.signal });
    if (!res.ok) {
      if (res.status === 429)
        throw new Error('API limit reached. Try again later.');
      throw new Error('Failed to reach geocoding service.');
    }
    const data = await res.json();
    clearTimeout(timeoutId);
    if (!data.results || data.results.length === 0)
      throw new Error('City "${city}" not found.');
    return data.results[0];  // { name, latitude, longitude, ... }
  } catch (error) {
    if (error.name === 'AbortError')
      throw new Error('Request took too long.');
    throw error;             // bubble up to handleSearch()
  }
}
```

Concept: async/await

fetch() takes time. 'await' pauses THIS function only; the page stays responsive. 'async' marks the function as pausable.

Concept: timeout via AbortController

If the API hangs, abort() fires at 5s and fetch throws AbortError - the user gets a clear message instead of an endless spinner.

Concept: HTTP status codes

res.ok means status 200-299. 429 = rate limited. Each failure becomes a human sentence via throw new Error(...).

Concept: errors bubble up

This function never touches the UI. It throws; handleSearch() catches and decides what the user sees. Data code and UI code stay separate.

Data shape example

Input

"Lagos"

→

Geocoding API JSON

results[0] = { name: 'Lagos', latitude: 6.45, longitude: 3.39, country: 'Nigeria' }

script.js Piece by Piece (3/4): fetchWeather() - API JSON to Dashboard

fetchWeather() condensed (lines 107-227)

```
const url = '...open-meteo.com/v1/forecast?latitude=${lat}
  &longitude=${lon}&daily=...&hourly=...&current=...&timezone=auto';

const data = await response.json();
if (!data.current || !data.daily || !data.hourly)
  throw new Error("Weather data is missing from the response.");

// write numbers into named DOM hooks
outdoorTemp.innerHTML  = '${Math.round(current.temperature_2m)}<span>C</span>';
humidity.textContent   = '${Math.round(current.relative_humidity_2m)}%';
windVal.textContent    = current.wind_speed_10m.toFixed(1);

// UV number -> label + dot position on the gradient bar
let uvText = 'Low';  let uvPos = 10;
if (uv >= 3 && uv <= 5) { uvText = 'Moderate'; uvPos = 30; }  // ...etc
uvDot.style.left = '${uvPos}%';

// WMO weather code -> icon name + label
const wmoMap = { 0: {label:'Clear', icon:'sun'},
                 61: {label:'Rain', icon:'cloud-rain'}, ... };

// build the 7-day forecast rows
for (let i = 0; i < 7; i++) {
  const info = wmoMap[data.daily.weather_code[i]] || {label:'Cloudy'};
  list.innerHTML += '<div class="forecast-item">...${temp}C</div>';
}
lucide.createIcons();   // new <i data-lucide> tags -> real SVGs
setUIState('success');  // reveal the dashboard
```

Concept: one URL, three datasets

The forecast API returns current (now), daily (7 days), hourly (per hour) in one JSON reply. Query params pick the fields.

Concept: validate before painting

Check data.current/daily/hourly exist BEFORE touching the DOM. A half-updated dashboard is worse than an error screen.

Concept: WMO codes

Weather services worldwide use standard numbers: 0=clear, 61=rain, 95=storm. wmoMap is a lookup table turning codes into icons.

Concept: template rows

The forecast list is built as an HTML string in a loop, then icons are re-scanned once at the end - one createIcons() call, not seven.

script.js Piece by Piece (4/4): Search Flow + Autocomplete Debounce

handleSearch() - the conductor (lines 230-260)

```
searchBtn.addEventListener('click', handleSearch);
cityInput.addEventListener('keypress', e => {
  if (e.key === 'Enter') handleSearch();
});

async function handleSearch() {
  const city = cityInput.value.trim();
  if (!city) { showError('Please enter a city name.'); return; }
  if (!navigator.onLine) { setUIState('error', 'offline'); return; }
  setUIState('loading');
  try {
    const geo = await getCoordinates(city);
    await fetchWeather(geo.latitude, geo.longitude, geo.name);
  } catch (err) {
    showError(err.message); setUIState('error', err.message);
  }
}
```

Autocomplete debounce (lines 270-278)

```
cityInput.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);            // cancel pending call
  const query = e.target.value.trim();
  if (query.length < 2) { dropdown.style.display='none'; return; }
  debounceTimer = setTimeout(() => fetchSuggestions(query), 300);
});
```

Concept: guard clauses

handleSearch() checks the cheap failures first (empty input, offline) and returns early. The happy path reads top-to-bottom.

Concept: debounce

Typing 'London' = 6 keystrokes. Without debounce: 6 API calls. With it: each keystroke resets a 300ms timer; only the pause after the last key fires ONE call.

Concept: click-outside close

A document-level click listener hides the dropdown unless the click was inside .search-container (checked with closest()).

Debounce timeline

ONE API call

fetchSuggestions('London') shows up to 5 clickable matches.

Cheat Sheet: Every Function + Key Vocabulary

One row per function in script.js. Find the line, read the job, jump into the source.

Function	Lines	Job
`updateTime()`	`11-21`	Clock + date header. Re-runs every second via setInterval.
`theme toggle`	`24-30`	Flips body.light-mode class; CSS variables restyle everything.
`showError()`	`33-38`	Slides in the toast, auto-hides after 4000ms.
`setUIState()`	`41-76`	The screen owner: empty / loading / error / success.
`getCoordinates()`	`79-104`	City name to lat/lon via geocoding API. 5s timeout.
`fetchWeather()`	`107-227`	Forecast API to DOM: temp, rain, UV, wind, sunrise, 7-day list. 8s timeout.
`handleSearch()`	`237-260`	The conductor: validate, set loading, geocode, fetch, catch errors.
`fetchSuggestions()`	`280-294`	Asks geocoding API for up to 5 city matches.
`showSuggestions()`	`296-311`	Builds clickable dropdown rows; click fills input and searches.
`click-outside`	`313-317`	Document listener closes the dropdown via closest().

Remember the pipeline

User input -> API JSON -> validated data -> DOM updates. Every feature in this app is one trip through that pipe.

async / await

Pause one function while data loads; page stays responsive.

debounce

Reset a timer per keystroke; act only after the typing pause.

guard clause

Check cheap failures first, return early, keep happy path flat.

state machine

One function decides the visible screen. No scattered show/hide.

AbortController

Cancel handle for fetch; turns a hang into a clear timeout error.

WMO code

Standard weather number (0=clear, 61=rain, 95=storm) mapped to icons.